



Universidad Centroamericana
José Simeón Cañas

BASE DE DATOS

Grupo F

Escenario A –

Sistema de Gestión Clínica Veterinaria

INTEGRANTES:

CESAR MELQUISEDEC ALVARADO LANDAVERDE

REYNALDO ALEXANDER AVELAR LINARES

LUIS ALESSANDRO CAMPOS MENJIVAR

DIANA ABIGAIL CASTILLO RAMOS

DAVID ALEJANDRO CRUZ CABALLERO

AXEL ALESSANDRO RIVAS MORAN

Fecha de entrega: viernes 19 de junio de 2026.

Contenido

Descripción del escenario:	3
Reglas de negocio:	3
Diagrama Entidad-Relación:	3
Esquema relacional normalizado:	4
Diccionario de datos de la base de datos:	4
Tabla propietario:	4
Tabla especie:	5
Tabla mascota	5
Tabla veterinario:	6
Tabla especialidad:	7
Tabla hoja_de_vida:	7
Tabla hoja_vida_especialidad	8
Tabla cita:	8
Tabla diagnóstico:	9
Tabla tratamiento:	10
Tabla medicamento:	10
Tabla tipo_procedimiento:	11
Tabla procedimiento:	11
Tabla factura:	12
Tabla det_factura:	13
Tabla tratamiento_medicamento	14
Tabla mascota_medicamento_alergia	15
Tabla diagnostico_procedimiento	15
Tabla factura_procedimiento	16
Tabla factura_medicamento	17
Consultas más frecuentes:	18
Documentación de funciones:	22
Procedimientos almacenados:	23
Disparadores creados:	24

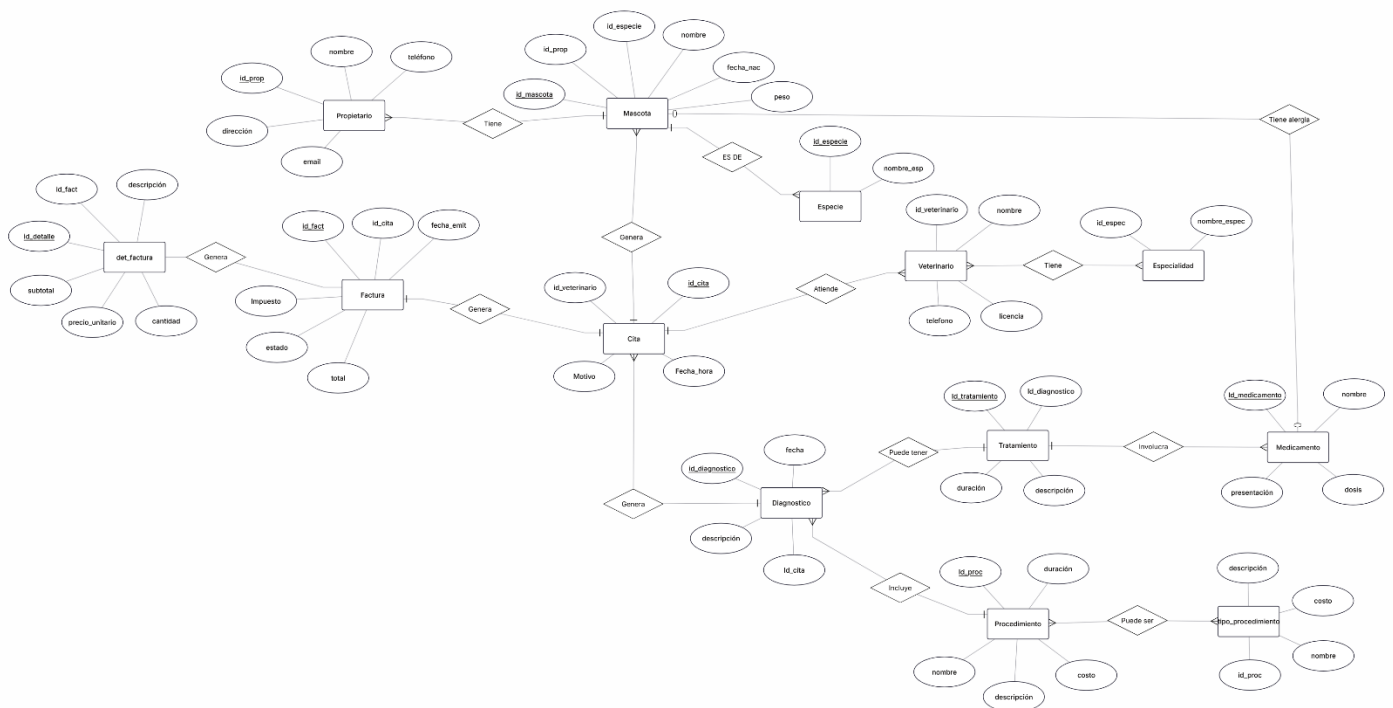
Descripción del escenario:

Una clínica veterinaria requiere un sistema para gestionar a sus pacientes (mascotas), propietarios, médicos veterinarios, citas, tratamientos y facturación.

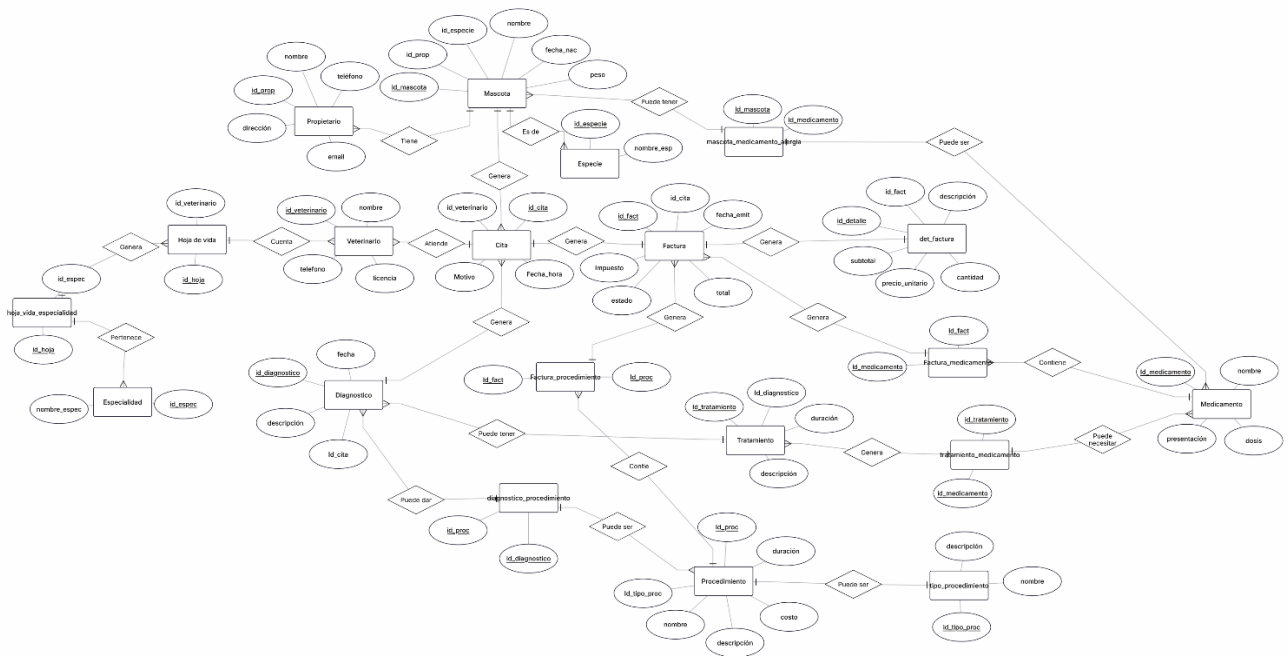
Reglas de negocio:

- Un propietario puede tener varias mascotas.
- Cada mascota pertenece a una especie y puede tener múltiples citas.
- Una cita es atendida por un veterinario y genera un diagnóstico con tratamientos.
- Los tratamientos pueden involucrar medicamentos.
- Cada cita genera una factura.
- Procedimiento y medicamento dejan facturación.

Diagrama Entidad-Relación:



Esquema relacional normalizado:



Diccionario de datos de la base de datos:

Tabla propietario:

Descripción: Tabla que se encarga de almacenar los datos del propietario de las mascotas, su Id, nombre, teléfono, dirección y email.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_prop	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3, 4
Nombre	Varchar (100)	No	No	Si	-	Ej: Carlos Alberto Mendoza
Teléfono	Varchar (20)	No	No	No	-	Ej: 7102-3344

Dirección	Varchar (200)	No	No	No	-	Ej: Colonia Escalón, Pasaje 3, #124, San Salvador
Email	Varchar (100)	No	No	No	-	Ej: juan.p1@email.com

Tabla especie:

Descripción: Tabla que se encarga de almacenar todas las especies a las que puede pertenecer las mascotas que se atienden.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_especie	Serial	Si	No	Si	-	Llave Primaria Ej: 1, 2, 3, 4
Nombre_esp	Varchar (50)	No	No	Si	-	Ej: Perro, Gato, Loro Real

Tabla mascota

Descripción: Cuenta con los campos para guardar la información con respecto a la mascota o mascotas de un propietario, como nombre, fecha de nacimiento, peso y su especie.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_mascota	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3
Id_prop	Integer	No	Si Propietario. Id_prop	Si	-	Llave foránea Ej: 1, 2, 3

						On delete cascade.
Id_espec	Integer	No	Si Especie. Id_especie	Si	-	Llave foránea Ej: 1, 2, 3, 4 On delete cascade.
Nombre	VarVarchar (50)	No	No	Si	-	Ej: Luna
Fecha_nac	Date	No	No	No	-	Ej: 2022-03-15 Formato de fecha en 'DD/MM/YYYY'
Peso	Decimal (8,2)	No	No	No	-	Ej: 15.5

Tabla veterinario:

Descripción: almacena los datos de los veterinarios que pertenecen a la clínica, para que sea más fácil el encontrar su información principal, teniendo un Id, nombre, licencia y teléfono.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_vet	Serial	Si	No	Si	-	Llave Primaria Ej: 1, 2, 3, 4
Nombre	Varchar (100)	No	No	Si	-	Ej: Dr.Francisco Alvarenga
Licencia	Varchar (100)	No	No	Si	-	Unique

						Ej: VET-2026-001
Telefono	Varchar (20)	No	No	No	-	Ej: 7022-1144

Tabla especialidad:

Descripción: tabla que almacena las diferentes especialidades con las que pueden contar los veterinarios de la clínica.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_espec	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3, 4, 5
Nombre_espec	Varchar (100)	No	No	Si	-	Ej: Medicina General

Tabla hoja_de_vida:

Descripción: tabla que se encarga de almacenar la hoja de vida del veterinario, con su Id propio y el ID veterinario, para que no hayan repeticiones.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_hoja	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3, 4
Id_veterinario	Integer	No	Si	Si	-	Unique

			Veterinario. Id_vet			Ej: 1, 2, 3, 4 On delete cascade
--	--	--	------------------------	--	--	--

Tabla hoja_vida_especialidad

Descripción: tabla intermediaria que une la tabla de hoja de vida con la de especialidades, para tener un mayor control y orden.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_hoja	Integer	Si	Si Hoja_de_vida. Id_hoja	Si	-	Llave primaria compuesta Ej: 1, 2, 3, 4
Id_espec	Integer	Si	Si Especialidad. Id_espec	Si	-	Llave primaria compuesta Ej: 1, 2, 3, 4

Tabla cita:

Descripción: tabla principal de la base de datos, la cual se encarga de almacenar los datos de las citas que se hayan realizado, cada una teniendo un Id individual, el Id de quien atendió, fecha y hora y motivo de su visita, genera las principales consultas.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_cita	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3, 4

Id_vet	Integer	No	Si Veterinario. Id_vet	Si	-	Ej: 5, 12, 23, 45 On delete cascade
Id_mascota	Integer	No	Si Mascota. Id_mascota	Si		Ej: 1, 2, 5, 8 On delete cascade
Fecha_hora	Timestamp	No	No	Si	-	Ej: Formato Fecha 'YYYY/MM/DD' 2026-01-10 08:30:00.000
Motivo	Varchar (250)	No	No	No		Ej: Consulta general

Tabla diagnóstico:

Descripción: tabla que almacena el diagnostico que genera la cita , teniendo un Id único, con su fecha y descripción correspondiente.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_diagnostico	Serial	Si	No	Si	-	Llave Primaria Ej:
Id_cita	Integer	No	Si Cita. Id_cita	Si	-	Ej: On delete cascade
Fecha	Date	No	No	No	-	Ej: Formato de fecha en 'DD/MM/YYYY'

Descripción	Text	No	No	No	-	Ej: Paciente sano
-------------	------	----	----	----	---	-------------------

Tabla tratamiento:

Descripción: se encarga de almacenar el tratamiento dado por el veterinario en su diagnóstico, describiendo y dejando claro su duración.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_tratamiento	Serial	Si	No	Si	-	Llave Primaria Ej: 1, 2, 3, 4
Id_diagnostico	Integer	No	Si Diagnostico. Id_diagnostico	Si	-	Ej: 1, 2, 3, 4 On delete Cascade
Descripción	Text	No	No	No	-	Ej: mantener balance calórico actual
Duración	Varchar (100)	No	No	No	-	Ej: mantenimiento permanente

Tabla medicamento:

Descripción: tabla que se encarga de almacenar el medicamento recetado durante el tratamiento.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_medimento	Integer	Si	No	Si	-	Ej: 1, 2, 3, 4

Nombre	Varchar (100)	No	No	Si	-	Ej: Meloxicam
Presentación	Varchar (100)	No	No	No	-	Ej: Suspensión oral
Dosis	Varchar (100)	No	No	No	-	Ej: 0.1 mg/kg

Tabla tipo_procedimiento:

Descripción: tabla que se encarga de almacenar los tipos de procedimientos disponibles que se pueden realizar en la clínica.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_tipo_proc	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3, 4
Nombre	Varchar (100)	No	No	Si	-	Ej: Consulta General
Descripción	Text	No	No	No	-	Ej: Evaluación clínica

Tabla procedimiento:

Descripción: almacena la información del procedimiento a realizar en el paciente, con su duración y costo.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_proc	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3, 4
Id_tipo_proc	Integer	No	Si Tipo_procedimiento. Id_tipo_proc	Si	-	Ej: 1, 2, 3, 4 On delete cascade
Nombre	Varchar (100)	No	No	Si	-	Ej: Consulta general
Descripción	Text	No	No	No	-	Ej: Evaluación clínica
Duración	Varchar (100)	No	No	Si	-	Ej: 30 minutos
Costo	Decimal (10, 2)	No	No	Si	-	Ej: 25

Tabla factura:

Descripción: otra de las principales tablas, se encarga de realizar la facturación de la cita correspondiente, manteniendo un Id único por cada una.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_fact	Serial	Si	No	Si	-	Llave Primaria

						Ej: 1, 2, 3, 4
Id_cita	Integer	No	Si Cita. Id_cita	Si	-	Ej: 1, 2, 3, 4 On delete cascade
Fecha_emit	Date	No	No		-	Ej: 2026-01-10 Formato de fecha en 'YYYY/MM/DD'
Total	Decimal (10,2)	No	No	Si	-	Ej: 35
Estado	Varchar (50)	No	No	No	-	Ej: Pagada
Impuesto	Decimal (10, 2)	No	No	No	-	Ej: 4.55

Tabla det_factura:

Descripción: detalla mas a fondo lo que contiene la factura que se ha pedido.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_detalle	Serial	Si	No	Si	-	Llave primaria Ej: 1, 2, 3, 4
Id_fact	Integer	No	Si Factura. Id_fact	Si	-	Ej: 1, 2, 3, 4 On delete cascade
Descripción	Text	No	No	No	-	Ej: Consulta veterinaria

Cantidad	Integer	No	No	Si	-	Ej: 1
Precio_unitario	Decimal (10,2)	No	No	Si	-	Ej: 25
Subtotal	Decimal (10, 2)	No	No	Si	-	Ej: 25

Tabla tratamiento_medimento

Descripción: tabla intermedia que se encarga de unir la tabla tratamiento, con el medicamento correspondiente de la tabla medicamento.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_tratamiento	Integer	Si	Si Tratamiento. Id_tratamiento	Si	-	Llave primaria compuesta Ej: 1, 2, 3, 4 On delete cascade
Id_medimento	Integer	Si	Si Medimento. Id_medimento	Si	-	Llave primaria compuesta Ej: 12, 5, 22, 8 On delete cascade

Tabla mascota_medicamento_alergia

Descripción: tabla de vital importancia que se encarga de mantener un orden en los medicamentos que generan alergias en los pacientes, para evitar problemas a la hora de recetárselos.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_mascota	Integer	Si	Si Mascota. Id_mascota	Si	-	Llave primaria compuesta Ej:1, 2, 3, 4 On delete cascade
Id_medicamento	Integer	Si	Si Medicamento. Id_medicamento	Si	-	Llave primaria compuesta Ej:1, 2, 3, 4 On delete cascade

Tabla diagnostico_procedimiento

Descripción: tabla intermedia que se encarga de unir diagnostico con procedimiento para llevar un orden claro de lo que se realizara.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_diagnostico	Integer	Si	Si Diagnostico. Id_diagnostico	Si	-	Llave primaria compuesta

						Ej: 1, 2, 3, 4 On delete cascade
Id_proc	Integer	Si	Si Procedimiento Id_proc	Si	-	Llave primaria compuesta Ej: 12, 5, 41, 3 On delete cascade

Tabla factura_procedimiento

Descripción: encargada de llevar un orden de la facturación del procedimiento.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_fact	Integer	Si	Si Factura. Id_fact	Si	-	Llave primaria compuesta Ej:1, 2, 3, 4 On delete cascade
Id_proc	Integer	Si	Si Procedimiento. Id_proc	Si	-	Llave primaria compuesta Ej: 1, 6, 3, 17 On delete cascade

Tabla factura_medicamento

Descripción: encargada de llevar un orden de la facturación del medicamento.

Campo	Tipo de dato	PK?	FK?	Obligatorio	Check	Comentario
Id_fact	Integer	Si	Si Factura. Id_fact	Si	-	Llave primaria compuesta Ej: 6, 7, 8, 9 On delete cascade
Id_medicamento	Integer	Si	Si Medicamento. Id_medicamento	Si	-	Llave primaria compuesta Ej: 12, 3, 8, 14 On delete cascade

Consultas más frecuentes:

```
--1. Historial clínico completo de una mascota
--util para: Ver todo lo que ha pasado con un paciente en una sola vista.

SELECT m.nombre AS mascota, c.fecha_hora, d.descripcion AS diagnostico, t.descripcion AS tratamiento
FROM mascota m
JOIN cita c ON m.id_mascota = c.id_mascota
JOIN diagnostico d ON c.id_cita = d.id_cita
JOIN tratamiento t ON d.id_diagnostico = t.id_diagnostico
WHERE m.nombre = 'Mota';

select id_mascota from cita;
select nombre from mascota where id_mascota = 12; -- lo dejare por si quiere consultar alguna mascota

--2. Facturación total por propietario
--util para: Saber cuánto ha gastado un cliente en total en la clínica.

SELECT p.nombre AS propietario, SUM(f.total) AS gasto_total
FROM propietario p
JOIN mascota m ON p.id_prop = m.id_prop
JOIN cita c ON m.id_mascota = c.id_mascota
JOIN factura f ON c.id_cita = f.id_cita
GROUP BY p.nombre;

--3. Listado de mascotas con sus medicamentos recetados
--util para: Revisar el historial de medicamentos de un paciente específico.

SELECT m.nombre AS mascota, med.nombre AS medicamento, med.dosis
FROM mascota m
JOIN cita c ON m.id_mascota = c.id_mascota
JOIN diagnostico d ON c.id_cita = d.id_cita
JOIN tratamiento t ON d.id_diagnostico = t.id_diagnostico
JOIN tratamiento_medicamento tm ON t.id_tratamiento = tm.id_tratamiento
JOIN medicamento med ON tm.id_medimento = med.id_medimento;

--4. Vets y sus especialidades
--util para: Saber qué médico es el mejor para asignar una cita según la especialidad necesaria.

SELECT v.nombre AS veterinario, e.nombre_espec AS especialidad
FROM veterinario v
JOIN hoja_de_vida hv ON v.id_veterinario = hv.id_veterinario
JOIN hoja_vida_especialidad hve ON hv.id_hoja = hve.id_hoja
JOIN especialidad e ON hve.id_espec = e.id_espec;

--5. Alertas de alergias
--util para: Evitar accidentes al recetar un medicamento.

SELECT m.nombre AS mascota, med.nombre AS medicamento_prohibido
FROM mascota m
JOIN mascota_medimento_alergia mma ON m.id_mascota = mma.id_mascota
JOIN medicamento med ON mma.id_medimento = med.id_medimento;
```

```
--6. Procedimientos realizados en una fecha específica
--util para: Reporte diario de actividades de la clínica.
```

```
SELECT m.nombre AS mascota, proc.nombre AS procedimiento, f.fecha_emit
FROM mascota m
JOIN cita c ON m.id_mascota = c.id_mascota
JOIN factura f ON c.id_cita = f.id_cita
JOIN factura_procedimiento fp ON f.id_fact = fp.id_fact
JOIN procedimiento proc ON fp.id_proc = proc.id_proc
WHERE f.fecha_emit = '2026-01-20';
```

```
select fecha_hora from cita;
```

```
--7. Citas pendientes por veterinario
--util para: Organizar la agenda diaria de los médicos.
```

```
SELECT v.nombre AS veterinario, m.nombre AS paciente, c.fecha_hora, c.motivo
FROM veterinario v
JOIN cita c ON v.id_veterinario = c.id_veterinario
JOIN mascota m ON c.id_mascota = m.id_mascota
ORDER BY c.fecha_hora ASC;
```

```
--8. Ingresos por tipo de procedimiento
--util para: Saber qué servicio deja más dinero a la clínica.
```

```
SELECT tp.nombre AS tipo_procedimiento, SUM(p.costos) AS ingresos_totales
FROM tipo_procedimiento tp
JOIN procedimiento p ON tp.id_tipo_proc = p.id_tipo_proc
JOIN factura_procedimiento fp ON p.id_proc = fp.id_proc
GROUP BY tp.nombre;
```

```
--9. Inventario de medicamentos usados
--util para: Saber qué medicamentos se usan más y necesitan reabastecimiento.
```

```
SELECT med.nombre, COUNT(fm.id_med medicamento) AS veces_facturado
FROM medicamento med
JOIN factura_med medicamento fm ON med.id_med medicamento = fm.id_med medicamento
GROUP BY med.nombre
ORDER BY veces_facturado DESC;
```

```
--9. Inventario de medicamentos usados
--util para: Saber qué medicamentos se usan más y necesitan reabastecimiento.
SELECT med.nombre, COUNT(fm.id_medicamento) AS veces_facturado
FROM medicamento med
JOIN factura_medicamento fm ON med.id_medicamento = fm.id_medicamento
GROUP BY med.nombre
ORDER BY veces_facturado DESC;

--10. Propietarios con más de una mascota
--util para: Campañas de marketing o fidelización.

SELECT p.nombre AS propietario, COUNT(m.id_mascota) AS cantidad_mascotas
FROM propietario p
JOIN mascota m ON p.id_prop = m.id_prop
GROUP BY p.nombre
HAVING COUNT(m.id_mascota) > 1;
```

-----CONSULTAS RECOMENDADAS POR EL INGENUERO-----

```
--11. Mascotas más atendidas por mes
--Útil para: Identificar temporadas de alta demanda de consultas.
```

```
SELECT m.nombre AS mascota,
       EXTRACT(MONTH FROM c.fecha_hora) AS mes,
       COUNT(c.id_cita) AS total_citas
FROM mascota m
JOIN cita c ON m.id_mascota = c.id_mascota
GROUP BY m.nombre, mes
ORDER BY mes, total_citas DESC;
```

```
--12. Veterinario con mayor número de citas en el trimestre
--Útil para: Evaluación de rendimiento y carga de trabajo.
```

```
SELECT v.nombre AS veterinario, COUNT(c.id_cita) AS total_citas
FROM veterinario v
JOIN cita c ON v.id_veterinario = c.id_veterinario
WHERE c.fecha_hora >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY v.nombre
ORDER BY total_citas DESC
LIMIT 1; -- todos tienen 1 ;)
```

--13. Medicamentos prescritos con mayor frecuencia
--Útil para: Gestión de inventario y compras a proveedores.

```
SELECT med.nombre AS medicamento, COUNT(tm.id_medicamento) AS veces_prescrito
FROM medicamento med
JOIN tratamiento_medicamento tm ON med.id_medicamento = tm.id_medicamento
GROUP BY med.nombre
ORDER BY veces_prescrito DESC;
```

--14. Ingresos totales por especialidad veterinaria
--Útil para: Saber qué ramas de la medicina veterinaria son más rentables para la clínica.

```
SELECT e.nombre_espec AS especialidad, SUM(f.total) AS ingresos_totales
FROM especialidad e
JOIN hoja_vida_especialidad hve ON e.id_espec = hve.id_espec
JOIN hoja_de_vida hv ON hve.id_hoja = hv.id_hoja
JOIN veterinario v ON hv.id_veterinario = v.id_veterinario
JOIN cita c ON v.id_veterinario = c.id_veterinario
JOIN factura f ON c.id_cita = f.id_cita
GROUP BY e.nombre_espec
ORDER BY ingresos_totales DESC;
```

--15. Propietarios cuyas mascotas no han asistido a citas en los últimos 6 meses
--Útil para: Campañas de marketing de "recuperación de clientes" (recordatorios de vacunas o chequeos).

```
SELECT DISTINCT p.nombre AS propietario, p.email
FROM propietario p
JOIN mascota m ON p.id_prop = m.id_prop
WHERE NOT EXISTS (
    SELECT 1
    FROM cita c
    WHERE c.id_mascota = m.id_mascota
    AND c.fecha_hora >= CURRENT_DATE - INTERVAL '6 months'
);
```

Documentación de funciones:

```
CREATE OR REPLACE FUNCTION obtener_historial_clinico(p_id_mascota INT)
RETURNS TABLE (
    fecha_cita TIMESTAMP,
    motivo_consulta TEXT,
    veterinario_atendio VARCHAR(100),
    diagnostico_descripcion TEXT,
    tratamiento_indicado TEXT,
    medicamento_recetado VARCHAR(100),
    procedimiento_realizado VARCHAR(100)
)
AS $$
BEGIN
    -- Verificar primero si la mascota existe
    IF NOT EXISTS (SELECT 1 FROM mascota WHERE id_mascota = p_id_mascota) THEN
        RAISE EXCEPTION 'La mascota con ID % no existe en la base de datos.', p_id_mascota;
    END IF;

    RETURN QUERY
    SELECT
        c.fecha_hora,
        c.motivo,
        v.nombre AS veterinario_atendio,
        d.descripcion AS diagnostico_descripcion,
        t.descripcion AS tratamiento_indicado,
        COALESCE(m.nombre, 'Ninguno') AS medicamento_recetado,
        COALESCE(p.nombre, 'Ninguno') AS procedimiento_realizado
    FROM cita c
    JOIN veterinario v ON c.id_veterinario = v.id_veterinario
    LEFT JOIN diagnostico d ON c.id_cita = d.id_cita
    LEFT JOIN tratamiento t ON d.id_diagnostico = t.id_diagnostico
    LEFT JOIN tratamiento_medicamento tm ON t.id_tratamiento = tm.id_tratamiento
    LEFT JOIN medicamento m ON tm.id_medicamento = m.id_medicamento
    LEFT JOIN diagnostico_procedimiento dp ON d.id_diagnostico = dp.id_diagnostico
    LEFT JOIN procedimiento p ON dp.id_proc = p.id_proc
    WHERE c.id_mascota = p_id_mascota
    ORDER BY c.fecha_hora DESC; -- Ordenado desde la más reciente a la más antigua
END;
```

```

--funcion contar mascotas que tiene propietario
CREATE OR REPLACE FUNCTION fn_cantidad_mascotas(
    p_id_prop INTEGER
)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_cantidad INTEGER;
BEGIN

    SELECT COUNT(*)
    INTO v_cantidad
    FROM mascota
    WHERE id_prop = p_id_prop;

    RETURN v_cantidad;

END;
$$;

SELECT fn_cantidad_mascotas(3);

```

```

--funcion alergias registradas de mascotas
CREATE OR REPLACE FUNCTION fn_tiene_alergias(
    p_id_mascota INTEGER
)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
BEGIN

    RETURN EXISTS (
        SELECT 1
        FROM mascota_medicamento_alergia
        WHERE id_mascota = p_id_mascota
    );

END;
$$;

```

Procedimientos almacenados:

```

--Revertir un pago que no se realizo correctamente
Create or replace PROCEDURE p_revertir_pago (id_factura int)
Language plpgsql
as $$
declare id_facturita int;
BEGIN
Select id_fact
into id_facturita
from factura
where id_fact = id_factura;

if id_facturita is null then
raise exception 'Id % de factura no encontrado', id_factura;
end if;

--borramos los detalles de factura
delete from det_factura
where id_fact = id_factura;

--actualizamos la factura con datos desde 0
update factura
set estado = 'En espera',
total = 0.00,
impuesto = 0.00
where id_fact = id_factura;
raise notice 'Factura de Id % puesta en espera con exito', id_factura;
end; $$;

```

```

--Creación de factura base para cita
Create or replace PROCEDURE factura_vacia(id_cita int)
Language plpgsql
as $$
BEGIN
--Iniciamos una factura vacia
    Insert into factura (id_cita, fecha_emit, total, estado, impuesto)
    Values (id_cita, CURRENT_DATE, 0.00, 'Pendiente', 0.00);

    Raise notice 'Factura iniciada con exito para la cita de Id %', id_cita;

    EXCEPTION WHEN OTHERS THEN
        Raise exception 'Error al crear la factura base. Motivo: %', SQLERRM;
end; $$;

```

Disparadores creados:

```

--funcion del trigger calcular subtotal
create or replace function fn_calcular_subtotal()
returns trigger
language plpgsql
as $$
begin
    new.subtotal := new.cantidad * new.precio_unitario;
    return new;
end;
$$;

--trigger calcular_subtotal
create trigger tr_calcular_subtotal
before insert on det_factura
for each row
execute function fn_calcular_subtotal();

```

```

--funcion del trigger de validacion de peso positivo
create or replace function fn_validar_peso_positivo()
returns trigger
language plpgsql
as $$
begin
    if new.peso < 0 then
        new.peso := new.peso * -1;
    end if;
    return new;
end;
$$;

--trigger validar peso positivo
create trigger tr_validar_peso_positivo
before insert on mascota
for each row
execute function fn_validar_peso_positivo();

```

```

--funcion del trigger de alerta de mascota alergica
create or replace function fn_alerta_mascota_alergica()
returns trigger
language plpgsql
as $$
declare
    v_id_mascota integer;
begin
    SELECT c.id_mascota
    INTO v_id_mascota
    FROM tratamiento t
    JOIN diagnostico d
    ON t.id_diagnostico = d.id_diagnostico
    JOIN cita c
    ON d.id_cita = c.id_cita
    WHERE t.id_tratamiento = NEW.id_tratamiento;

    IF EXISTS (
        SELECT 1
        FROM mascota_medicamento_alergia mma
        WHERE mma.id_mascota = v_id_mascota
        AND mma.id_medicamento = NEW.id_medicamento
    ) THEN

        RAISE EXCEPTION
        'La mascota es alergica a este medicamento, cambiar medicamento';
    end if;
    return new;
end;
$$;

```